Token-First Long-Term Memory for LLM Coding Agents: A Two-Budget Design, Paired Retrieval Evaluation, and a Three-Tier Measurement of Token Cost

Lawrence Eldridge

6 July 2026 (version 2.1 — renamed to claude-engram)

Abstract

Retrieval-augmented memory for large-language-model coding agents is usually costed in bytes, yet the model consumes only text tokens, so byte-efficiency and token-efficiency are distinct budgets that a naive design conflates. This paper specifies claude-engram, a local-first memory and code index for Claude Code that separates the two, and evaluates it with paired statistics on a 297-fact, 244-query corpus mined from real captured sessions. The default semantic backend, bge-base with int8 vectors, beat the lexical stub by 3.1 times on Recall@1 (0.463 against 0.148, McNemar $p < 0.001$) and, with the pairing the second version of the corpus makes possible, beat its smaller sibling decisively (paired delta +0.066, $p = 0.033$), resolving a hypothesis the first version of this paper left open. Retrieval-specialised and larger alternatives failed to beat the default, and Matryoshka truncation to a third of the dimensions left recall statistically unchanged at one third of the bytes. Token cost was measured at three tiers: a conservative usage ledger, a trace-driven counterfactual replay of 287 recorded sessions (79,263 tokens saved on verifiable search sweeps, a 95% reduction), and a live paired A/B over 48 headless sessions, which found a significant cost, not a saving, on single-turn tasks answerable from static context (+20,925 tokens per task, $p = 1e\text{-}6$). The token-savings claim is therefore conditional on task shape, and the paper states the boundary rather than hiding it.

Contents

1 Introduction

A coding agent's context window is finite, and every token placed in it has a cost in money, latency, and displaced attention. An agent that "remembers everything" is therefore not an option: memory has to be selective, and it has to be cheap at the moment of use. The problem this paper addresses is how to give a coding agent durable, cross-session memory whose cost is paid in the currency the model actually consumes, namely text tokens, rather than in the bytes a storage-centred design optimises. The first version of this paper established the design and a retrieval evaluation at n = 77 queries; this second version widens the corpus to n = 244, adds paired statistics that resolve comparisons the first version could only call indicative, evaluates four alternative embedding models against the default, and adds the measurement the first version named as its main gap: what the system actually does to tokens spent per task.

1.1 Related work

Three strands inform the design. The first is human memory research. The Atkinson and Shiffrin (1968) multi-store model separates a small, capacity-bounded short-term store from a durable long-term store, with rehearsal as the transfer mechanism; the Ebbinghaus forgetting curve describes retention decaying with time unless reinforced; Roediger and Karpicke (2006) show that retrieval itself, not only repetition, strengthens a memory (the testing effect); and the active-systems-consolidation account describes an offline pass that replays, integrates, and prunes traces. The second strand is retrieval systems: dense-vector similarity search, retrieval-augmented generation, approximate-nearest-neighbour indexes, quantisation of embeddings, and Matryoshka representation learning, which trains a vector's leading dimensions to stand alone so that truncation degrades gracefully. The third is software architecture: Fowler's Patterns of Enterprise Application Architecture and the Cosmic Python extensions, in

particular Repository, Gateway with a Separated Interface, the Functional Core and Imperative Shell split, and Command Query Responsibility Segregation (CQRS). For the measurement method, the paper borrows trace-driven simulation from the systems literature of the 1980s and 1990s: when live experiments are expensive, recorded workloads replayed through a model of the alternative system yield real-workload evidence at near-zero cost.

1.2 The gap

Prior approaches leave three gaps for the coding-agent setting. First, always-on retrieval exposed as a model-callable tool carries a standing token cost and hands the model an agency decision on every turn, whether or not memory is relevant. Second, raw vector similarity recalls stale and conflicting facts, because similarity alone has no model of recency, reinforcement, or contradiction. Third, the literature optimises storage footprint, which is the wrong budget: shrinking bytes does not shrink the tokens injected into the context window, and the two can even move in opposite directions. A fourth gap belongs to evaluation practice rather than design: memory systems are usually justified by retrieval quality alone, and the end-to-end token cost of running one, including the fixed cost of its tool schemas and injections, is rarely measured at all.

1.3 The present study

claude-engram is a local-first memory and code/docs index for Claude Code, packaged as a plugin, that closes the design gaps by construction. Capture runs off the interactive path and distils transcripts into atomic facts. Recall runs as a hook, not a tool, so it has zero standing token cost and no model agency; it injects only the facts that clear a similarity gate, capped at a fixed character budget. A cognitive-memory lifecycle re-ranks and retires facts by recency, reinforcement, and contradiction rather than by similarity alone. The core runs on the Python standard library, with semantic embeddings and language-model distillation as optional adapters, so the default install works with no network and no API key. This version of the paper closes the evaluation gap as well: it measures token cost at three tiers of rising strength and rising expense, and reports the negative result the strongest tier produced alongside the positive results of the weaker two.

1.4 Claims

This paper advances one design claim and six measurable hypotheses. H1 to H3 are carried forward from version 1; H4 to H6 are new in version 2.

Design claim. Treating recall as a threshold-gated hook, and treating byte-efficiency and token-efficiency as separate budgets, keeps interactive token cost near zero while retaining useful memory.

H1. int8 quantisation of the embedding vectors does not materially reduce retrieval quality relative to full-precision float, at personal-store scale.

H2. Embedding model size is a larger recall lever than vector precision: moving from a smaller to a larger model changes Recall@1 more than moving from float to int8.

H3. A larger labelled corpus narrows the confidence interval on the retrieval metrics, and paired per-query tests resolve between-backend differences that independent intervals cannot.

H4. Neither retrieval-specialised nor larger general-purpose embedding models beat the incumbent default (bge-base) on this corpus, on either the memory or the code-index surface.

H5. For a Matryoshka-trained model, truncating stored vectors to one third of their native dimensions leaves retrieval quality statistically unchanged.

H6. Using the plugin reduces the tokens spent completing a task in Claude Code.

---

2 Method

2.1 Corpus and environment

Retrieval is measured with a labelled paraphrase set held in `plugins/engram/bench/dataset.json` and executed by `plugins/engram/bench/run_eval.py`. Each query is worded away from its target fact so that lexical overlap alone is insufficient, which stresses semantic matching. The version 2 corpus has 297 facts and 244 queries, of which 56 queries have two or more gold facts and 50 facts are hard negatives that no query targets. It was built by mining the author's live memory store (1,837 active facts captured from real working sessions on this repository) through `plugins/engram/bench/mine_corpus.py`, which applies a contamination filter (facts about the benchmark, its metrics, or its measurement machinery are excluded, so the corpus cannot leak its own labels), privacy flaggers, and near-duplicate removal, followed by a mandatory human review of every candidate. Identifying strings were replaced with neutral placeholders. The project was renamed from claude-ltm to claude-engram (plugin ltm to engram) after these measurements; the dataset, method, and figures are unchanged, and the paths and commands below reflect the renamed tree. The version 1 corpus (64 facts, 77 queries) is frozen verbatim as `plugins/engram/bench/dataset-v1.json`, so the figures published in version 1 remain reproducible. Mined facts describe one real codebase and its development history, which makes the corpus a harder discrimination task than version 1: many facts are near neighbours about the same subsystems. Absolute scores are therefore lower than version 1 across every backend, and the two versions' absolute numbers are not comparable; comparisons within version 2 are.

The runtime is CPython 3.12 on macOS, with `fastembed` 0.8.0 providing the semantic backends. Every figure in this paper was reproduced in one session at commit `57528b8`.

2.2 Variables and conditions

The retrieval experiments manipulate the embedding backend (`hash`, a dependency-free lexical stub, against six `fastembed` semantic models), vector precision (int8 against float), and Matryoshka truncation (native 768 dimensions against the leading 256). The dependent variables are Recall@1, Recall@3, MRR@10, and bytes per fact, with per-query latency recorded as an operational cost. Controlled factors are the search path (the same quantised cosine scan for every quantised run), the dataset, and the caps applied during evaluation (`top_k` and `min_sim`

are relaxed so ranking is measured over the full set). Query fairness across asymmetric models is controlled: the adapter routes queries through fastembed's `query_embed`, so each model receives its own trained query prefix (`plugins/engram/core/adapters/fastembed_gw.py`).

The token-cost experiment (tier 3) manipulates a single factor, plugin enabled against plugin disabled, within task: each of 24 lookup tasks about this repository runs twice in fresh headless sessions. The dependent variable is a price-weighted composite of the session's token usage (input at 1.0, cache-write at 1.25, cache-read at 0.1, output at 1.0, matching the provider's price ratios), with answer correctness as a gate.

2.3 System and instruments

Architecture. claude-engram is CQRS with a hexagonal (ports and adapters) core. The write side and the read side have opposite performance profiles and are separated accordingly. Capture is heavy, batched, and latency-tolerant; it runs detached at the `Stop`, `SessionEnd`, and `PreCompact` hooks. Recall is tiny and latency-critical; it runs in `UserPromptSubmit` for just-in-time injection and at `SessionStart` for a stable project core. The POEAA roles are fixed: memory access is a Repository (`core/store.py`), never Active Record; the embedding provider is a Gateway behind a Separated Interface (`core/ports/embedding.py`, `core/adapters/`); distillation, ranking, scoring, and quantisation are pure functions in the Functional Core; and the injected payload is a one-line-per-fact data-transfer object rendered by `core/recall/__init__.py::render_block`, which returns the empty string on no match (a Null Object, so irrelevant turns cost nothing).

Ranking. Retrieval is a hybrid re-rank, not raw similarity. Each candidate that clears the similarity gate `min_sim` receives a priority score combining similarity, recency decay, and a frequency boost, computed by the pure `core/recall/__init__.py::_score` over `core.domain.scoring`. Superseded facts are excluded at the SQL layer, so a replaced fact cannot resurface, and the injected block is bounded by a character cap so the token budget is fixed regardless of store size.

Storage and truncation. Each fact is stored as its text plus a quantised embedding: an int8 vector as the primary search representation and binary sign bits for a fast Hamming pre-filter. An opt-in configuration key, `embedding_truncate_dim` (default 0, off), keeps only the leading N dimensions and re-normalises before quantisation (`core/adapters/fastembed_gw.py::truncate_renorm`); it is only meaningful for Matryoshka-trained models, and changing it invalidates stored vectors.

Retrieval instrument. The benchmark embeds each query, ranks the corpus through the quantised search path, and scores the position of the gold facts. Recall@k is the fraction of queries with a gold fact in the top k; MRR@10 is the mean reciprocal rank within the top ten. Version 2 adds paired inference: per-query outcomes are retained, backends evaluated on identical queries are compared with an exact McNemar test on the discordant pairs (`bench/run_eval.py::mcnemar_exact`), and MRR deltas carry a seeded percentile-bootstrap 95% interval (`bench/run_eval.py::bootstrap_ci`, 10,000 resamples, fixed seed). Pairing cancels between-query variance, so smaller deltas are resolvable than the independent Wilson intervals suggest. A separate 30-query instrument (`bench/eval_code_index.py`) measures the code-index sur-

face: natural-language queries against gold symbol anchors, over a fresh index of the plugin's own tree built per backend.

Token-cost instruments (three tiers). Tier 1 is the usage ledger, `usage_summary` in `core/service.py`: the cost side records every injected byte; the savings side records only *measured* events, a targeted symbol or section read priced as the whole file minus the returned span. The headline figure excludes heuristic estimates and prices the session core under the prompt-cache model (a 1.25 times write premium plus a 0.1 times re-read per turn, with recall events as the turn proxy), taking the larger of the raw and cache-adjusted cost views. Tier 2 is trace-driven replay (`bench/replay.py`, driven by the `engram replay` CLI): recorded session transcripts are parsed for search sweeps, defined as two or more consecutive Grep, Glob, or whole-file Read calls ending on a whole-file Read; each sweep's actual token cost is the byte size of its tool results at four bytes per token; the counterfactual re-queries the live index with the sweep's own search terms and credits a saving only when the returned hits verifiably contain the file the sweep landed on, pricing the indexed path as the search outline plus the fetched symbol body plus a fixed per-call overhead. Bounded reads earn nothing, sweeps without a landing file earn nothing, and unverified hits earn nothing, so the credit is a floor. Tier 3 is the live paired A/B (`bench/run_ab.py`): each task runs once with the plugin enabled and once with it disabled via a settings override, the disabled arm is verified plugin-free by scanning its transcript for plugin activity (schema or hook references outside attachment lines), pairs failing the answer check in either arm are reported separately rather than dropped, and significance uses an exact Wilcoxon signed-rank test on the paired composites (`bench/run_ab.py::wilcoxon_exact`).

2.4 Protocol

A captured session becomes memory by the sequence distil, embed, quantise, and persist, off the interactive path. A query becomes an injection by the sequence embed, cosine scan over the int8 vectors, similarity gate, hybrid re-rank, and top-k render under the character cap. The retrieval benchmark runs this path for every query and aggregates the metrics; the paired section prints for every backend pair evaluated in the same run. The replay protocol iterates every transcript recorded for this repository, extracts sweeps, and totals actual against counterfactual cost. The A/B protocol runs the 24 tasks sequentially, alternating arms within task, and reads usage from the CLI's JSON result. Exact commands are given in § 3.7.

---

3 Results

3.1 Data cleaning

No retrieval queries were excluded, and every backend under test loaded and ran. Each `fastembed` model was warmed with one throwaway embedding before timing so that the one-off model load is excluded from per-query latency. In the A/B experiment, the first run's off-arm verification produced 24 false positives from a substring scan that matched repository paths in legitimate answer content; the checker was corrected to detect plugin *activity* only, the off arms were re-verified clean (no plugin tool schemas, calls, or hook injections outside attachment lines), and the statistics below were computed from the recorded per-session log of that run. No ses-

sions were re-run and no pairs were dropped: all 24 pairs completed and every answer passed its correctness check in both arms.

3.2 Descriptive statistics: retrieval on the version 2 corpus

All seven backends were evaluated on the same 297-fact, 244-query corpus in this session (int8 storage throughout; the float twin of the default appears in § 3.4).

Backend	Dim	Recall@1	Recall@3	MRR@10	Bytes/fact	Query ms
hash (lexical stub)	256	0.148	0.234	0.210	288	8.5
bge-small-en-v1.5	384	0.398	0.611	0.518	432	15.4
bge-base-en-v1.5 (default)	768	**0.463**	**0.656**	**0.574**	864	29.3
arctic-embed-s	384	0.348	0.467	0.430	432	15.1
arctic-embed-m	768	0.324	0.443	0.403	864	29.1
nomic-embed-text-v1.5	768	0.422	0.635	0.547	864	31.1
mxbai-embed-large-v1	1024	0.475	0.680	0.587	1152	48.2

Wilson 95% intervals on Recall@1 (n = 244): hash [0.109, 0.197]; bge-small [0.338, 0.460]; bge-base [0.402, 0.526]; arctic-s [0.291, 0.410]; arctic-m [0.268, 0.385]; nomic [0.362, 0.485]; mxbai [0.414, 0.538].

3.3 Paired inference: the model comparisons

Paired per-query tests on the identical 244 queries give the between-backend findings their force. Four results follow.

Semantic against lexical is decisive. bge-base against hash: paired Recall@1 delta +0.316 (discordant pairs 15 against 92, McNemar $p < 0.001$), MRR delta +0.364 with bootstrap interval [+0.300, +0.426]. The relative Recall@1 gain is 3.1 times (0.463 against 0.148).

The model-size question version 1 left open is resolved. bge-base against bge-small: paired Recall@1 delta +0.066 (discordant 17 against 33, $p = 0.033$), MRR delta +0.056 [+0.017, +0.096]. The independent intervals overlap, as they did in version 1; the pairing resolves what they cannot. Recall@3 trends the same way without reaching significance (+0.045, $p = 0.099$).

Retrieval-specialised models lose here. arctic-embed-s against bge-base: Recall@1 delta -0.115 ($p = 0.001$), Recall@3 delta -0.189 ($p < 0.001$). arctic-embed-m fares worse still (-0.139, $p < 0.001$). Notably, a 77-query screening pass run before the corpus was widened had ranked arctic-s *above* bge-small; on the full corpus that ordering reversed and arctic-s sits significantly below bge-small on Recall@3 and MRR ($p < 0.001$). Small-corpus screening results did not survive scale, which is the concrete case for H3.

A larger general model does not earn its cost. mxbai-embed-large-v1 against bge-base: Recall@1 delta +0.012 (discordant 19 against 22, $p = 0.755$), MRR delta +0.013 [-0.025, +0.052]. At 2.3 times the query latency and 1.33 times the bytes per fact, an insignificant delta does not justify a default change. nomic-embed-text-v1.5 is likewise statistically indistinguishable from the default (Recall@1 delta -0.041, $p = 0.229$) and is retained only as the Matryoshka candidate (§ 3.4).

3.4 Precision and truncation

int8 against float (H1). On the version 2 corpus the default backend's int8 run scores 0.463 Recall@1 against the float run's 0.459, with Recall@3 identical (0.656) and a single discordant query out of 244 ($p = 1.0$); the MRR delta is -0.003. The int8 representation gives up nothing measurable, at 864 against 3,072 bytes per fact, roughly a quarter of the float footprint. Version 1's exact null is thus replicated on a corpus 3.2 times larger.

Matryoshka truncation (H5). nomic-embed-text-v1.5 truncated to its leading 256 dimensions against its native 768, paired on the same queries: Recall@1 0.402 against 0.422 (delta -0.020, discordant 17 against 12, $p = 0.458$), Recall@3 0.602 against 0.635 ($p = 0.185$), MRR delta -0.026 [-0.054, +0.002]. No difference reaches significance, while stored bytes fall 3 times (864 to 288 per fact) and query latency halves (31.1 to 14.7 ms). The lever ships as `embedding_truncate_dim`, default off, because the default model is not Matryoshka-trained.

3.5 The code-index surface

On the 30-query code-retrieval set (natural-language query to gold symbol anchor), the general default beat the code-specialised model: bge-base Recall@1 0.733 [0.556, 0.858], Recall@3 0.867; jina-embeddings-v2-base-code 0.700 [0.521, 0.833], 0.800. The paired delta is -0.033 with 2 against 1 discordant pairs ($p = 1.0$): no evidence for a per-surface model, and the point estimate favours the incumbent. The indexed text for a code symbol already carries its qualified name, signature, and docstring, which appears to suit a general text model; $n = 30$ is small, but the direction rules out the hypothesised advantage.

3.6 Token cost at three tiers

Tier 1, the ledger (observational floor). On this machine's live store at the time of measurement: 142 injections costing about 9,700 tokens raw (about 10,700 under the cache-adjusted view), against about 554,600 tokens of measured savings from 59 targeted reads and 45 bounded reads, a measured-only net of about 543,900 tokens. This figure is observational, accumulated over weeks of real use, and its savings side counts only priced events (whole-file reads avoided), so it is a floor rather than an estimate.

Tier 2, trace-driven replay (real workloads, counterfactual). Across all 287 transcripts recorded for this repository, the replay found 30 residual search sweeps with an actual cost of 212,729 tokens. Eight sweeps were creditable under the conservative rules (the index answer verifiably contained the landing file); their indexed-path cost was 3,700 tokens against the actual cost of those sweeps, a saving of 79,263 tokens, or a 95% reduction on the creditable set. Two caveats bound this figure. The recorded history is post-deployment, so the plugin's own enforcement had already suppressed most sweeps; pre-deployment history would show more. And 22 of 30 sweeps earned nothing because verification failed or the landing file was not in the index's answer, which the credit rules count as zero rather than guessing.

Tier 3, the live paired A/B (strongest design, negative result). Over 24 lookup tasks about this repository, each run in a fresh single-turn headless session with the plugin on and off (48 sessions), every answer was correct in both arms, and the plugin arm cost *more*: mean composite 56,117 against 35,192 tokens per task, a paired difference of +20,925 tokens per task against the plugin (exact Wilcoxon $p = 0.000001$; the plugin arm was cheaper on 2 of 24 tasks; median paired difference -18,869). The mechanism is visible in the per-task traces: a near-constant overhead of roughly 19,000 composite tokens in the plugin arm, which is the plugin's tool schemas and session-start injections priced at the cache-write premium, in sessions that end after one turn and so never re-read the cached prefix at the discounted rate. The tasks, chosen to be answerable in both arms, were answerable from the repository's always-loaded instructions in both arms, so retrieval had no savings to make.

Read together, the three tiers give H6 a conditional verdict rather than a simple one, and § 4.1 states it.

3.7 Reproducibility

All retrieval figures were produced from the repository root at commit `57528b8`, against `plugins/engram/bench/dataset.json` (297 facts, 244 queries), with `fastembed` 0.8.0:

```
python3 plugins/engram/bench/run_eval.py --backends "hash,fastembed@BAAI/bge-small-en-v1.5,\
fastembed@BAAI/bge-base-en-v1.5,fastembed@snowflake/snowflake-arctic-embed-s,\
fastembed@snowflake/snowflake-arctic-embed-m,fastembed@nomic-ai/nomic-embed-text-v1.5,\
fastembed@mixedbread-ai/mxbai-embed-large-v1"
```

The truncation comparison uses the spec suffix `%256` (`fastembed@nomic-ai/nomic-embed-text-v1.5%256`); the precision comparison appends `+float`; the code-index experiment is `python3 bench/eval_code_index.py` from `plugins/engram/`. The replay is `python3 bin/engram replay`; the A/B is `python3 bench/run_ab.py` (note that it spends real API tokens; `--dry-run` prints the plan). The version 1 figures remain reproducible against the frozen `plugins/engram/bench/dataset-v1.json`. Raw output for every experiment in this section is in Appendix A.

---

4 Discussion

4.1 Hypothesis evaluation

H1 (int8 approximately equals float) is supported. Quantisation loss remains negligible on the harder corpus, so the compact representation is kept and a float-rescore stage remains not worth building.

H2 (model size beats precision) is supported, now with significance. Version 1 could state the direction only; the paired test on the widened corpus puts bge-base above bge-small at $p = 0.033$ on Recall@1 while the precision delta stays at zero.

H3 (a larger corpus and pairing resolve finer differences) is supported, twice over. The Recall@1 interval half-width fell from roughly 0.10 at $n = 77$ to roughly 0.06 at $n = 244$, and the paired tests resolved both the model-size question and the elimination of the arctic models. The sharpest illustration is the reversal: the small-corpus screening had ranked arctic-s above bge-small, and the full corpus reversed that ordering with $p < 0.001$.

H4 (no evaluated alternative beats the default) is supported. Two retrieval-specialised models sit significantly below the default; a model 33% larger by stored bytes is statistically indistinguishable from it; and the code-specialised model loses on the surface it specialises in. The incumbent default survives a four-way challenge it could have lost.

H5 (Matryoshka truncation is free at one third the size) is supported for the Matryoshka-trained candidate, with every paired delta inside noise and the operational costs falling by 3 times (storage) and 2 times (latency).

H6 (the plugin reduces tokens per task) receives a split verdict, and the split is the finding. On real recorded workloads containing search sweeps, the counterfactual saving is large (95% on creditable sweeps, tier 2) and the observational ledger agrees (tier 1). On single-turn tasks answerable from static context, the strongest experimental design available here found a significant net cost of about 21,000 composite tokens per task (tier 3). The apparatus that produces the savings, tool schemas and session-start injection, is a fixed investment priced at the cache-write premium; it amortises across the turns of a working session at the 0.1 times cache-read rate and repays itself when tasks actually require retrieval or search. In one-shot sessions over questions the repository's standing instructions already answer, the investment is pure overhead. The honest statement of the design claim is therefore conditional: near-zero *marginal* token cost per irrelevant turn, as designed, on top of a *fixed* per-session cost that requires session length or search-shaped work to recoup.

4.2 Integration

The int8 and truncation results together sit against the vector-store convention of storing full-precision embeddings: at personal-store scale, precision buys nothing measurable and, for a Matryoshka model, three quarters of the dimensions are similarly dispensable. The model bake-off aligns with a recurring finding in retrieval evaluation, that leaderboard-adjacent claims do not transfer automatically to a specific corpus and task: two models marketed on retrieval benchmarks lost to a two-year-old general baseline here. The tier-3 negative echoes the systems literature's caution about fixed overheads in measurement: the A/B deliberately isolated a single

session shape (one turn, fresh cache) and found exactly the case the amortisation model predicts the design loses. The trace-driven replay, borrowed from the cache-simulation tradition, is the bridge between the two: it measures the workload the design was built for, at zero marginal spend.

4.3 Limitations and threats to validity

Five limitations bound these results. First, the corpus, though 3.2 times larger than version 1, is single-project and single-author: the mined facts describe one repository's development, and generalisation to other codebases is untested. Second, the corpus was mined from the same store the system operates on, and although facts about the benchmark itself were excluded by filter and human review, subtler self-reference cannot be ruled out entirely. Third, the replay's credit rules are deliberately conservative and its history is post-deployment, so tier 2 understates savings in both directions of unknown magnitude. Fourth, the A/B covers one task shape (single-turn repository lookups), one repository, and one run of 24 pairs; it cleanly demonstrates the fixed-overhead case but does not measure the multi-turn, search-heavy case where tiers 1 and 2 indicate the savings live. Fifth, the ledger's token arithmetic uses a four-bytes-per-token heuristic and a modelled cache discount rather than provider-billed figures.

4.4 Implications and future directions

Three implications follow directly. A coding-agent memory should spend its budget on model choice and what is stored, not on vector precision, and should validate any model swap with paired tests on its own corpus rather than public leaderboards. Matryoshka truncation is a real storage and latency lever once the default model is Matryoshka-trained, and the configuration surface for it now exists. And token-savings claims for agent memory systems should be stated conditionally on task shape, because the fixed cost of the apparatus is significant and measurable.

The next measurement is the A/B the negative result calls for: multi-turn sessions and tasks that require cross-session memory or multi-file search, where the fixed cost amortises and retrieval has work to do. The inert working-memory levers reported in version 1 (gist chunking, a focus and activated-set split, a retrieval scaffold, use-feedback inhibition, and spreading activation) remain staged behind benchmark gates, and the widened corpus now gives those gates the resolution to detect the effects they are waiting for.

References

Source at HEAD (this repository, commit `57528b8`).

- Recall read path and hybrid re-rank: `plugins/engram/core/recall/__init__.py`.
- Capture service and usage ledger: `plugins/engram/core/service.py`.
- Memory store (Repository): `plugins/engram/core/store.py`.
- Embedding gateway and truncation: `plugins/engram/core/adapters/fastembed_gw.py`.

- Benchmark harness, paired statistics, and datasets: `plugins/engram/bench/run_eval.py`, `plugins/engram/bench/dataset.json`, `plugins/engram/bench/dataset-v1.json`.
- Corpus miner: `plugins/engram/bench/mine_corpus.py`.
- Trace-driven replay: `plugins/engram/bench/replay.py` (CLI: `bin/engram replay`).
- Live paired A/B: `plugins/engram/bench/run_ab.py`, `plugins/engram/bench/tasks.json`.
- Code-index experiment: `plugins/engram/bench/eval_code_index.py`.

Prior design records.

- Version 1 of this paper (6 July 2026; the n = 77 evaluation, reproducible against the frozen `plugins/engram/bench/dataset-v1.json` at commit `aa0bcc5`).
- Two-budget model, POEAA map, and cache analysis: `DESIGN.md`.
- Pattern map and layering: `.claude/rules/02-architecture/01-poeaa-and-layers.md`.

External literature.

- Atkinson, R. C., and Shiffrin, R. M. (1968). Human memory: a proposed system and its control processes.
- Ebbinghaus, H. (1885). Memory: a contribution to experimental psychology.
- Roediger, H. L., and Karpicke, J. D. (2006). Test-enhanced learning: taking memory tests improves long-term retention.
- Cowan, N. (2001). The magical number 4 in short-term memory.
- Kusupati, A., et al. (2022). Matryoshka representation learning.
- Fowler, M. (2002). Patterns of Enterprise Application Architecture.
- Percival, H., and Gregory, B. (2020). Architecture Patterns with Python.

Appendix A: Raw experimental output (this session, commit 57528b8)

A.1 Retrieval, seven backends, version 2 corpus (n = 244)

```
dataset: 297 facts, 244 paraphrased queries

backend                                      dim   recall@1  recall@3  mrr@10  embed_ms/fact  query_ms  bytes/fact
-------------------------------------------  ----  --------  --------  ------  -------------  --------  ----------
hash                                         256   0.148     0.234     0.210   0.233          8.514     288.0
fastembed@BAAI/bge-small-en-v1.5             384   0.398     0.611     0.518   3.369          15.447    432.0
fastembed@BAAI/bge-base-en-v1.5              768   0.463     0.656     0.574   6.498          29.348    864.0
fastembed@snowflake/snowflake-arctic-embed-s 384   0.348     0.467     0.430   2.993          15.078    432.0
fastembed@snowflake/snowflake-arctic-embed-m 768   0.324     0.443     0.403   8.369          29.059    864.0
fastembed@nomic-ai/nomic-embed-text-v1.5     768   0.422     0.635     0.547   11.770         31.093    864.0
fastembed@mixedbread-ai/mxbai-embed-large-v1 1024  0.475     0.680     0.587   22.417         48.204    1152.0

Selected paired comparisons (positive delta favours the second backend; * = p<0.05):
  hash -> bge-base:        drecall@1 +0.316 (15/92, p=0.000)*  dmrr +0.364 [+0.300, +0.426]*
  bge-small -> bge-base:   drecall@1 +0.066 (17/33, p=0.033)*  dmrr +0.056 [+0.017, +0.096]*
  bge-base -> arctic-s:    drecall@1 -0.115 (50/22, p=0.001)*  dmrr -0.144 [-0.199, -0.088]*
  bge-base -> arctic-m:    drecall@1 -0.139 (56/22, p=0.000)*  dmrr -0.171 [-0.231, -0.112]*
  bge-base -> nomic:       drecall@1 -0.041 (33/23, p=0.229)   dmrr -0.027 [-0.071, +0.016]
  bge-base -> mxbai-large: drecall@1 +0.012 (19/22, p=0.755)   dmrr +0.013 [-0.025, +0.052]
```

A.2 Matryoshka truncation (n = 244)

```
fastembed@nomic-ai/nomic-embed-text-v1.5      768  0.422  0.635  0.547  31.1ms  864 B/fact
fastembed@nomic-ai/nomic-embed-text-v1.5%256  256  0.402  0.602  0.521  14.7ms  288 B/fact
paired: drecall@1 -0.020 (17/12, p=0.458)  drecall@3 -0.033 (18/10, p=0.185)
        dmrr -0.026 [-0.054, +0.002]
```

A.3 int8 against float, default backend (n = 244)

```
backend                              dim  recall@1  recall@3  mrr@10  query_ms  bytes/fact
-----------------------------------  ---  --------  --------  ------  --------  ----------
fastembed@BAAI/bge-base-en-v1.5      768  0.463     0.656     0.574   29.0      864.0
```

```
fastembed@BAAI/bge-base-en-v1.5+float  768  0.459     0.656     0.571   20.6      3072

paired: drecall@1 -0.004 (discordant 1/0, p=1.000)  drecall@3 +0.000 (0/0, p=1.000)
        dmrr -0.003 [-0.008, -0.000]
```

A.4 Code-index surface (n = 30)

```
fastembed@BAAI/bge-base-en-v1.5:               recall@1 0.733 [0.556, 0.858]  recall@3 0.867
fastembed@jinaai/jina-embeddings-v2-base-code: recall@1 0.700 [0.521, 0.833]  recall@3 0.800
delta recall@1 (code model - general): -0.033 (discordant 2/1, McNemar p=1.000)
```

A.5 Trace-driven replay (287 transcripts)

```
transcripts : 287 files (0 unreadable)
sweeps      : 30 found, 8 creditable via the index
actual cost : ~212,729 tokens across all sweeps
indexed cost: ~3,700 tokens for the creditable sweeps
SAVED       : ~79,263 tokens (counterfactual floor, creditable sweeps only)
```

A.6 Live paired A/B (24 tasks, 48 sessions)

```
pairs: 24, all correct both arms: True
mean composite tokens/task: on=56,117  off=35,192  delta=+20,925 (on-arm overhead)
plugin cheaper on 2/24 tasks; Wilcoxon exact two-sided p=0.000001
median paired diff (off-on): -18,869
composite weights: input 1.0, cache-write 1.25, cache-read 0.1, output 1.0
```

A.7 Usage ledger snapshot (observational, this machine)

```
cost  - injected  : 142 injections, ~9,669 tokens (~10,676 cache-adjusted)
saved - measured  : 59 targeted reads + 45 bounded reads, ~554,556 tokens
saved - estimated : 10 recall shortcuts, ~12,000 tokens (heuristic, excluded from headline)
NET measured      : ~543,880 tokens
```

Appendix B: Selected configuration defaults

Key	Default	Meaning
`top_k`	3	facts injected per prompt (the injected focus)
`min_sim`	0.12	similarity gate for injection
`max_chars`	800	hard character cap per injection (the token guard)
`half_life_days`	30	recency-decay half-life
`supersede_threshold`	0.85	similarity at which a newer fact retires an older one
`embedding`	hash	default backend; `fastembed` enables semantic recall
`embedding_truncate_dim`	0	Matryoshka truncation (off); N keeps the leading N dims

The full configuration surface is documented in `README.md`.